SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section introduces the strategic technical solution for the Finsight platform, designed to solve the manual accounting bottlenecks faced by small business owners as outlined in the Product Review Documentation.

1.1. Purpose:

The system analysis document highlights the technical scope, architectural decisions, and specific AI integration strategies behind Finsight's development. The key elements of the system covered are as follows:

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
   Finsight operates on a modern Client-Server architecture. The frontend is built using React (Vite), communicating via RESTful APIs with a Node.js/Express backend. Data is persisted locally using SQLite for the MVP, and the core processing engine relies heavily on external AI services (GLM-5.1).
2. ***Data Flows:***
   How raw transaction data (CSV or manual input) flows from the user's browser, moves through the backend Express API, gets transmitted to the GLM API for intelligent categorization, returns as structured JSON, and is finally stored persistently in the SQLite database for dashboard rendering.
3. ***Model Process:***
   The model shows the end-to-end workflow of the core features: ingesting raw financial data, applying AI-driven categorization, interacting with a context-aware chatbot, and synthesizing data into an executive report.
4. *Role of Reference:*
   This document serves as a reference for all developers and testers to understand the high-level architecture, prompt management, and data handling of this project.

*1.2. **Background**:*

Small business owners and freelancers traditionally handle their finances via manual entry into spreadsheets or complex accounting software that requires dedicated financial knowledge.

1. ***Previous Version:***
   - *Previously, business owners had to act as their own bookkeepers—downloading bank statements, manually assigning categories (e.g., Rent, Utilities) row by row, and manually creating formulas to calculate runway and profit margins.*
2. ***Changes in Major Architectural Components:***

© UMHackathon 2026

- *Finsight replaces manual logic with an **AI Service Layer**. Instead of static categorization rules, it uses a Large Language Model to infer intent and context from raw transaction descriptions.*

3. ***New Capabilities:***
   - *Conversational querying of financial data. Instead of building pivot tables, users can simply ask, "Why did my expenses increase this month?" and receive real-time, data-backed answers.*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Small Business Owner	*Uploads transaction data, reviews the analytics dashboard, and chats with the AI Advisor.*	*Simple data upload experience, clear visual charts, and accurate, easy-to-understand financial reports.*
AI Advisor (System)	*Processes user inputs, categorizes data, and generates actionable recommendations.*	*High availability from the GLM API and accurately formatted JSON outputs.*
Development Team	*Builds the platform, writes system prompts, and maintains the backend API.*	*Clear API usage contracts, strict JSON mode enforcement, and modular codebase.*
QA Team	*Validates system behavior, especially the AI's math constraints and categorization logic.*	*Coverage of edge cases, proper fallback mechanisms when the AI hallucinates, and latency testing.*
Stakeholders	*Roles*	*Expectations*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Responsive Desktop/Mobile Browser)*
Architecture	*Client-Server API with AI Service Integration*

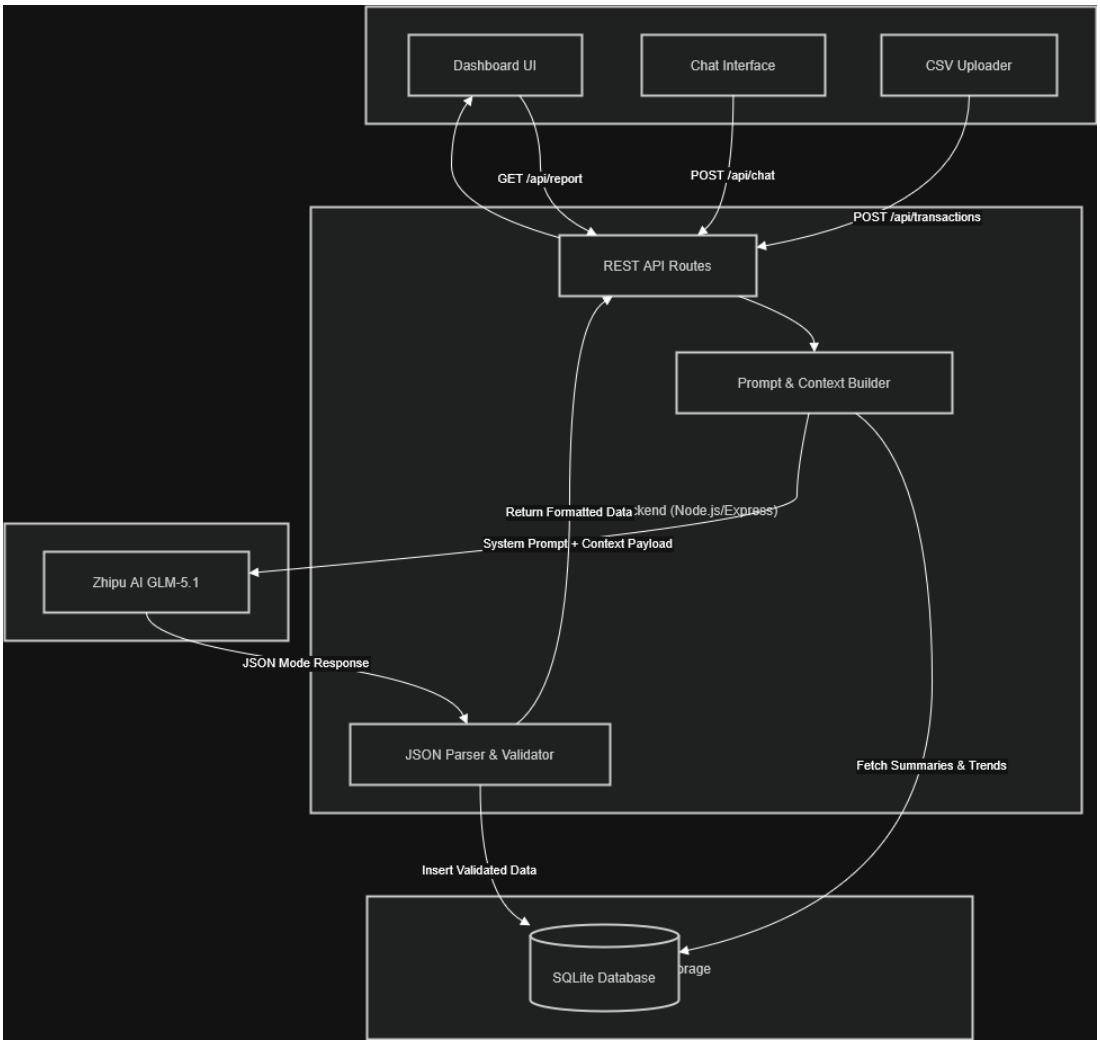
Finsight is structured as a client-server web system. The frontend client (React Dashboard) communicates with the backend API layer (Node.js/Express). The backend acts as the orchestrator: it manages local persistence in an SQLite database and acts as a secure proxy to communicate with the external LLM Service (ZAI GLM-5.1).

2.1.2 LLM as Service Layer

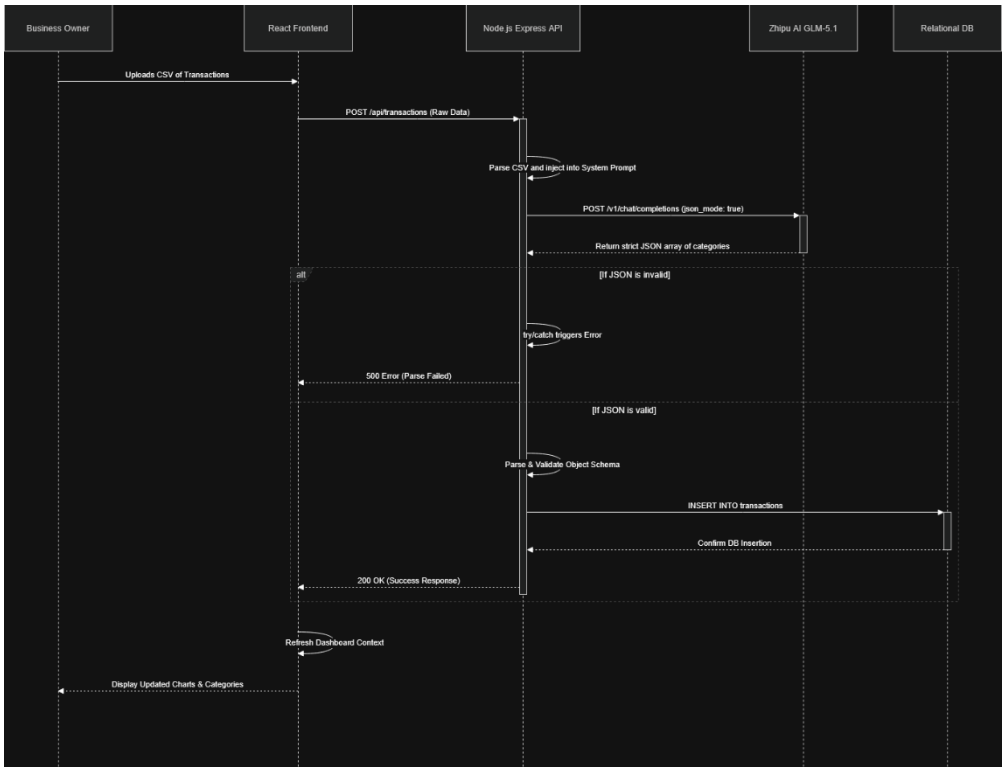
GLM is integrated as a core microservice layer responsible for data transformation and reasoning, rather than just a generic "chat" module. It serves three distinct backend functions: Categorization (POST /api/transactions), Conversational Analytics (POST /api/chat), and Report Synthesis (GET /api/report).

© UMHackathon 2026

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. Technological Stack

Frontend: *React (Vite) with Tailwind CSS and Shadcn UI – chosen for rapid, highly aesthetic UI prototyping and seamless client-side routing.*

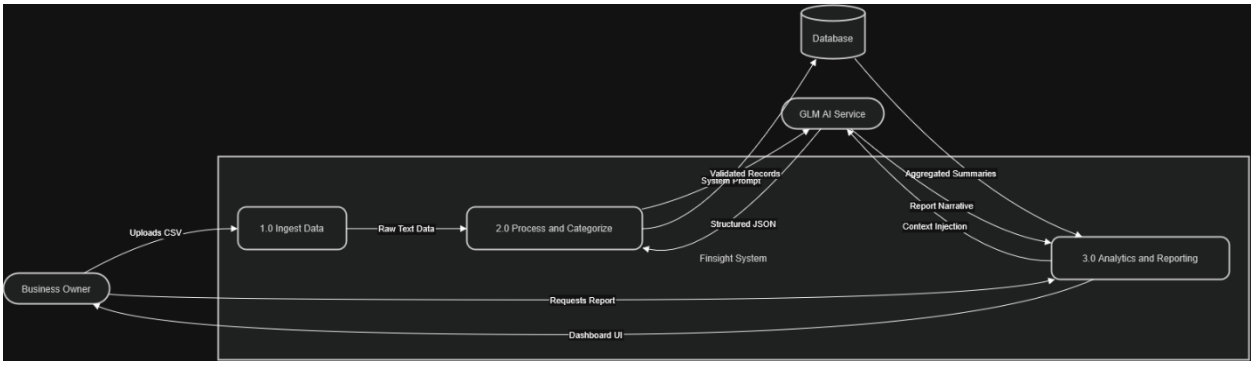
Backend: *Node.js (Express) – Lightweight RESTful API handling prompt orchestration and JSON parsing.*

Database: *SQLite – File-based relational database chosen for extreme portability and zero-config setup during MVP development.*

External AI: *ZAI ilmu-glm-5.1 – Core reasoning engine for classification and natural language generation.*

2.3. Key Data Flows

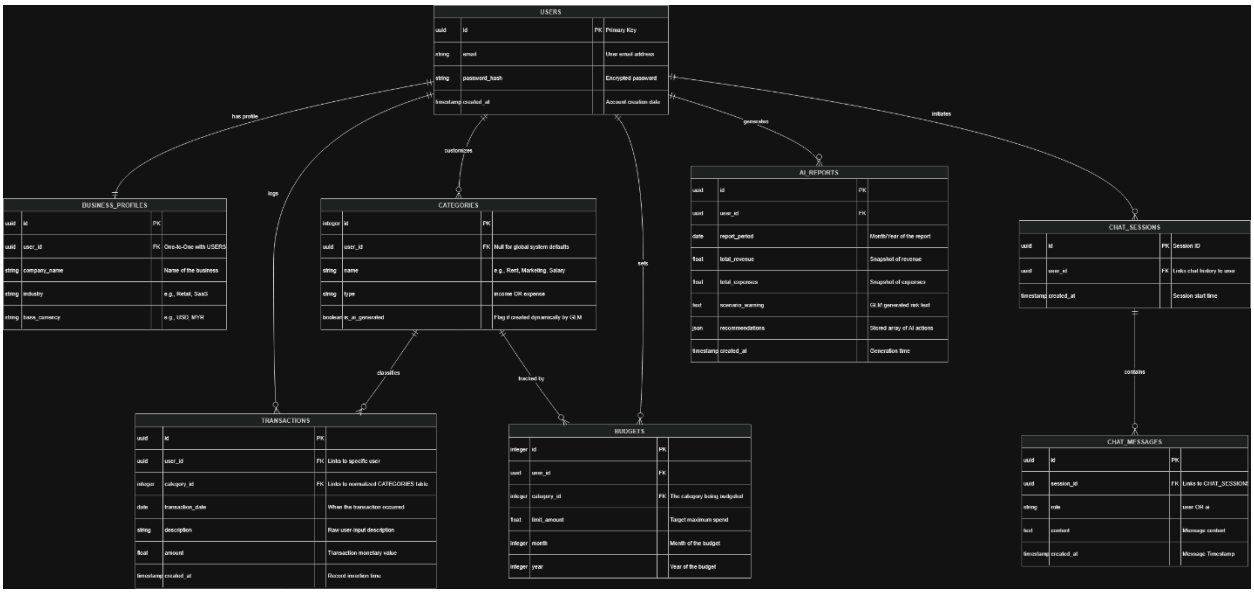
2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. **Normalized Database Schema**



3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

#	*Features*	*Description*
1	Smart Transaction Ingestion	Users can input raw transactions which the AI automatically categorizes into standard financial buckets (Rent, Revenue, Marketing, etc.).
2	Dashboard Analytics	A visual summary interface displaying Net Profit, Total Revenue, Total Expenses, and Category breakdowns using Recharts.
3	Conversational Advisor	A chatbot where users can ask specific questions about their finances, drawing live context from their actual database summaries.
4	Executive Report Generation	One-click generation of a structured financial report highlighting trends, issues, and actionable AI recommendations.

3.2. *Non-Functional Requirements (NFRs)*

Quality	Requirements	Implementation
Token Latency	*GLM categorization calls must return within an acceptable window to not freeze the UI.*	*Asynchronous API calls with loading states ("GLM is thinking..."). Backend implements a 3-second retry on 5xx errors.*
Reliability (Math)	*The system must never hallucinate core financial totals (Revenue/Profit).*	*Backend hardcodes actual DB aggregations into the final report payload, bypassing the LLM's math calculations entirely.*
Error Handling	*Invalid JSON from the LLM must not crash the backend.*	*Wrapping JSON.parse(content) in strict try/catch blocks. If parsing fails, the backend returns a safe fallback error to the user.*
Cost Efficiency	*Token usage per user session must remain tightly controlled.*	*Strict enforcement of max_tokens limit (1500 for categories, 3000 for reports) and injection of summaries instead of raw histories.*

3.3. Out of Scope / Future Enhancements

***Direct Bank API Integrations**: Auto-syncing with banks (e.g., Plaid integration) is deferred to future versions.*

***Multi-User & Multi-Entity Accounting**: Currently designed as a single-user application. Role-based access control (RBAC) and accountant portals are out of scope.*

***Tax Filing**: Automated generation and submission of state/federal tax forms.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: *Changes to System Prompts (e.g., adjusting the strictness of the JSON output or adding new categories) will be tested locally before updating the production environment. We will monitor parsing failure rates to ensure new prompts don't break the JSON structure.*

© UMHackathon 2026

4.1.2. ***Emergency Rollback (ER) & Golden Release****: If the GLM API experiences an outage or a breaking model update that causes persistent JSON parsing errors across user sessions, an ER is triggered to revert to the previous "Golden Release" prompt template or switch to a fallback LLM model.*

4.1.3. ***Priority Matrix:***

P1 Critical*: If the GLM API returns 500/504 errors for more than 3 consecutive retries, the system degrades gracefully, informing the user that the AI is temporarily unavailable while still allowing manual dashboard viewing.*

4.2. Monitoring:

- *The Node.js backend logs all GLM API response statuses.*
- *Any failures in the try/catch blocks for JSON.parse are flagged in the console, indicating that the LLM deviated from the enforced json_mode schema, which requires prompt tuning.*

4.3. Assumptions:

- ***User Internet:*** *Users maintain a stable internet connection while uploading data or requesting reports to prevent API timeouts.*
- ***Data Integrity:*** *The transaction amounts inputted by the user are accurate. The AI can reason about the data, but it cannot fix inherently wrong input numbers.*
- ***Single Environment:*** *The MVP assumes a single local environment deployment via Vite/Express.*

4.4. External Dependencies:

Tools	*Purpose*	*Risks*
Zhipu AI GLM-5.1 API	*Core reasoning engine handling categorization, multi-step inference, and structured report generation.*	***High*** *– API rate limits or model downtime will disable the categorization and chat features. Mitigation: Implement graceful UI error states indicating AI downtime.*
TailwindCSS & Shadcn	*Frontend styling and pre-built accessible components.*	***Low*** *– Packaged locally at build time.*

© UMHackathon 2026

SQLite3	*Local database persistence.*	***Medium*** *– File locking issues if multiple users access the local file concurrently. Mitigation: Future migration to PostgreSQL for scale.*

5. Project Management & Team Contributions

5.1. Project Timeline:

Days 1-2: Project Planning, Schema Design, and UI Wireframing.

Days 3-5: Frontend scaffolding (React, Tailwind) and Backend API initialization (Express, SQLite).

Days 6-8: GLM-5.1 Integration: Prompt engineering, JSON mode enforcement, and hooking the AI to the Chat/Dashboard interfaces.

Days 9-10: End-to-end testing, catching hallucination edge cases, and preparing the final presentation demo.

5.2. Team Members Role:

Lee Chee Cheng – Developer (Lead Frontend & Architecture)
Tan Lok Qi – Developer (UI/UX Implementation & Components)
Lee Chun Kit – Developer (Backend API & Database Schema)
Yap Wai Hong – Developer (AI Integration & Prompt Engineering)
Chin Guan Ee – Developer (AI Integration & Prompt Engineering)

5.3. Recommendations:

Database Migration*: For production readiness post-hackathon, we highly recommend migrating the database from local SQLite to a managed PostgreSQL instance (e.g., AWS RDS or Supabase) to support concurrent users and scalable cloud deployment.*

Caching Strategy: *Implement a caching layer (like Redis) for the Dashboard summary metrics to reduce unnecessary SQLite reads and prevent redundant calls to the GLM API if the financial context hasn't changed.*

© UMHackathon 2026